

INTERNATIONAL POSTCODE SYSTEM (IPCS)

par

Bartosz Cieřlicki

Rochdale 2018

Hypothèses du système

Le concept principal du système est de créer un système uniforme de codes postaux pour la Terre entière. L'objectif le plus important est :

ENCODER LES COORDONNÉES GÉOGRAPHIQUES DE CHAQUE CODE POSTAL DANS SON PROPRE ENREGISTREMENT.

Le système ainsi conçu éliminera le besoin d'une grande base de données mondiale contenant l'emplacement de chaque code postal. Il réduira ou éliminera également complètement les coûts de géocodage, tout en accélérant considérablement les systèmes informatiques recherchant une position géographique pour une adresse demandée.

Hypothèses :

- La zone identifiée par un code postal ne devrait pas dépasser 1000 m², afin de pouvoir atteindre facilement et précisément le lieu recherché, à l'aide d'une carte ou d'une navigation par satellite.
- Le système doit couvrir chaque partie de la planète, quels que soient la population, la densité du bâti, le relief ou le type de surface.
- Les codes postaux doivent être calculés / créés sur la base d'un algorithme, et non attribués arbitrairement.
- On peut calculer les coordonnées géographiques à partir de chaque enregistrement de code postal, qui indique le centre de la zone.

Le International Postcode System (IPCS) :

- devrait être basé sur l'alphabet latin, le plus répandu au monde.
- devrait comporter un nombre relativement restreint de caractères, permettant une mémorisation facile.
- ne devrait pas contenir de caractères spéciaux propres à un pays donné.
- devrait éliminer les erreurs de lecture et d'écriture les plus fréquentes (c'est-à-dire la lettre O confondue avec le chiffre 0, et la lettre I confondue avec le chiffre 1).

Construction des secteurs

Pour calculer le International Postcode System (IPCS), nous utilisons la latitude et la longitude en degrés décimaux. La longitude se situe dans l'intervalle $(-180, 180>$, tandis que la latitude se situe dans l'intervalle $<-90, 90>$. La toute première étape essentielle pour calculer le code postal international consiste à arrondir la latitude et la longitude au quart de 0,001.

Exemple :

Supposons que le nombre $t \in <0.000, 0.001>$, le nombre x étant notre arrondi. Si :

$$x \in <0.000, 0.000125) \rightarrow x = 0.000$$

$$x \in <0.000125, 0.000375) \rightarrow x = 0.00025$$

$$x \in <0.000375, 0.000625) \rightarrow x = 0.0005$$

$$x \in <0.000625, 0.000875) \rightarrow x = 0.00075$$

$$x \in <0.000875, 0.001> \rightarrow x = 0.001$$

Grâce à ces arrondis, la Terre entière est divisée en zones. Les coordonnées obtenues identifient chaque secteur créé sur Terre, tout en étant également son centre. Cela répond à l'hypothèse formulée au départ.

L'extrait de grille se présente ainsi :

La structure des secteurs basée sur les coordonnées fait que la superficie de chaque secteur, formé par l'arrondi de la latitude et de la longitude, diminue à mesure que l'on se rapproche des pôles de la Terre. Les plus grandes superficies se trouvent donc à l'équateur et avoisinent 728 m², ce qui répond à notre hypothèse.

Nous pouvons calculer le nombre de secteurs ainsi créés. Les nombres sont arrondis au quart d'un millièm, chaque millièm étant donc divisé en 4.

Pour la longitude, nous avons l'intervalle numérique $(-180, 180>$. Par x , nous désignons le nombre de divisions. Il vaut :

$$x = 360 * 1000 * 4 \quad (1)$$

Pour la latitude, nous avons l'intervalle numérique $<-90, 90>$. Par y , nous désignons le nombre de divisions. Il vaut :

$$y = 181 * 1000 * 4 \quad (2)$$

(La valeur 181, au lieu de 180, tient compte du fait que l'intervalle de latitude $<-90, 90>$ est fermé des deux côtés — le pôle Sud (-90°) et le pôle Nord (90°) sont deux points distincts — contrairement à l'intervalle de longitude $(-180, 180>$, qui n'est fermé que d'un seul côté, puisque -180° et 180° désignent le même méridien.)

En utilisant (1) et (2), nous calculons le nombre de secteurs pour la Terre entière :

$$\text{Number of sectors} = x * y = (360 * 1000 * 4) * (181 * 1000 * 4) = 1\,042\,560\,000\,000 \quad (3)$$

Il ne reste plus qu'à construire le Code Postal International (IPCS) pour chacun des secteurs formés/créés.

Construction du International Postcode System (IPCS)

En utilisant les lettres latines et les chiffres, nous disposons de 10 chiffres et de 26 lettres. Nous retirons de l'alphabet les lettres O et I (pour respecter les hypothèses du système). Nous obtenons ainsi les 34 caractères suivants : {0, 1, 2, 3, 4, 5, 6, 7, 8, 9, A, B, C, D, E, F, G, H, J, K, L, M, N, P, Q, R, S, T, U, V, W, X, Y, Z}. À partir de ces 34 caractères, nous créons un système numérique – un système en base 34. À gauche figurent les nombres dans le système en base 34, à droite leur équivalent dans le système décimal.

0 = 0	8 = 8	G = 16	Q = 24	Y = 32
1 = 1	9 = 9	H = 17	R = 25	Z = 33
2 = 2	A = 10	J = 18	S = 26	
3 = 3	B = 11	K = 19	T = 27	
4 = 4	C = 12	L = 20	U = 28	
5 = 5	D = 13	M = 21	V = 29	
6 = 6	E = 14	N = 22	W = 30	
7 = 7	F = 15	P = 23	X = 31	

Chaque nombre en base 34 peut être représenté dans le système décimal comme suit :

$$...dcba_{34} = f(a) \times 34^0 + f(b) \times 34^1 + f(c) \times 34^2 + f(d) \times 34^3 + ...$$

, où f est une fonction qui associe à chaque chiffre du système en base 34 son équivalent décimal (voir (4)). Ici, a, b, c, d sont des chiffres du système en base 34.

Exemple :

$$\begin{aligned} AGZ15Z_{34} &= f(Z) \times 34^0 + f(5) \times 34^1 + f(1) \times 34^2 + f(Z) \times 34^3 + f(G) \times 34^4 + f(A) \times 34^5 \\ &= 33 \times 34^0 + 5 \times 34^1 + 1 \times 34^2 + 33 \times 34^3 + 16 \times 34^4 + 10 \times 34^5 \\ &= 33 \times 1 + 5 \times 34 + 1 \times 1156 + 33 \times 39304 + 16 \times 1336336 + 10 \times 45435424 \\ &= 33 + 170 + 1156 + 1297032 + 21381376 + 454354240 = 477034007 \end{aligned}$$

Comme on peut le constater, l'écriture en base 34 est bien plus courte. Il nous faudra 8 caractères pour générer des codes postaux pour la planète entière. Le nombre maximal de caractères dans le système postal anglais et irlandais actuel est de 7, au Japon de 7 chiffres, aux Pays-Bas de 6 caractères, etc. Compte tenu de la longueur des codes postaux dans d'autres pays, et du fait que le International Postcode System est un système mondial, nous considérons que 8 caractères sont acceptables et répondent aux hypothèses de notre système.

Le plus grand nombre que nous puissions stocker avec 8 chiffres dans notre système en base 34 est (les calculs détaillés ont été omis) :

$$ZZZZZZZZ_{34} = 1\,785\,793\,904\,895$$

Il en résulte que la Terre pourrait avoir 1 785 793 904 896 codes postaux (ce nombre est supérieur d'une unité au plus grand nombre, car 00000000 peut également être un code). D'après le calcul précédent (3), nous connaissons le nombre de secteurs de notre structure, soit 1 042 560 000 000. Ce nombre est inférieur à 1 785 793 904 895 ; par conséquent, 8 chiffres en base 34 suffisent pour identifier sans ambiguïté chaque secteur. Les superficies des secteurs ne sont pas égales en raison de

cette structure. Les plus grandes se situent près de l'équateur et avoisinent 728 m². La superficie de la Terre est d'environ 510 100 000 km², soit 510 100 000 000 000 m². Pour calculer la superficie moyenne d'un secteur, nous effectuons une opération simple :

$$510\,100\,000\,000\,000\text{ m}^2 / 1\,042\,560\,000\,000 \approx 489.2764\text{ m}^2$$

Ainsi, ZZZZZZZZ₃₄ répond à nos hypothèses, et la superficie de chaque secteur est inférieure à 1000 m².

Il ne reste plus qu'à convertir le centre du secteur au format International Postcode System (IPCS).

Pour nos besoins, nous représentons la partie entière des nombres négatifs comme suit :

- Pour la longitude, la partie entière négative x est convertie en $360 + x$.
- Pour la latitude, l'entier négatif y est converti en $180 + y$.

La partie décimale reste inchangée. En résumé :

$$\text{Longitude } -XXX.xxxxx \rightarrow (360 + (-XXX)).xxxxx$$

$$\text{Latitude } -YYY.yyyyy \rightarrow (180 + (-YYY)).yyyyy$$

Exemples de conversion :

- Le nombre -75.43725 pour la longitude devient 285.43725, car $360 + (-75) = 285$.
- Le nombre -0.411250 pour la longitude devient 360.411250, car $360 + 0 = 360$.
- Le nombre -15.44375 pour la latitude devient 165.44375, car $180 + (-15) = 165$.

En créant l'algorithme, il faut tenir compte du fait que, dans cet enregistrement, le pôle Sud à 90 degrés de latitude et le pôle Nord à -90 degrés de latitude sont assimilés l'un à l'autre.

En tenant compte de la conversion des nombres négatifs et de l'arrondi, la forme générale des coordonnées enregistrées dans le système décimal est :

$$\text{Longitude: } XXX.xxxqq, \text{ where } XXX \in \{x \in \mathbb{N}: 000 \leq x \leq 360\}, xxx \in \{x \in \mathbb{N}: 000 \leq x \leq 999\}, qq \in \{00, 25, 50, 75\}$$

$$\text{Latitude: } YYY.yyygg, \text{ where } YYY \in \{x \in \mathbb{N}: 000 \leq x \leq 180\}, yyy \in \{x \in \mathbb{N}: 000 \leq x \leq 999\}, gg \in \{00, 25, 50, 75\}$$

À l'étape suivante, chaque élément des nombres XXX.xxxqq et YYY.yyygg est converti en système binaire.

$$XXX_{10} = XXXXXXXXX_2 \text{ — max. 360, requires 9 bits.}$$

$$xxx_{10} = xxxxxxxxx_2 \text{ — max. 999, requires 10 bits.}$$

$$qq_{10} = qq_2 \text{ — only 4 possible values: } 00_{10} = 00_2, 25_{10} = 01_2, 50_{10} = 10_2, 75_{10} = 11_2$$

$$YYY_{10} = YYYYYYY_2 \text{ — max. 180, requires 8 bits.}$$

$$yyy_{10} = yyyyyyyyy_2 \text{ — max. 999, requires 10 bits.}$$

$$gg_{10} = gg_2 \text{ — only 4 possible values: } 00_{10} = 00_2, 25_{10} = 01_2, 50_{10} = 10_2, 75_{10} = 11_2$$

Nous disposons ensuite les nombres binaires dans l'ordre suivant :

$$XXXXXXXXXX YYYYYYYY xxxxxxxxxx yyyyyyyyyy qq gg_2$$

Nous prenons un nombre d'au moins 41 bits, le mettons à zéro, puis y inscrivons les bits ci-dessus de droite à gauche en conservant leur ordre. Le nombre obtenu est égal à :

$$XXXXXXXXXXYYYYYYYxxxxxxxxxyyyyyyyyyqqgg_2$$

À la dernière étape, nous convertissons le nombre binaire en base 34. Le nombre en base 34 obtenu constitue également notre Code Postal International (IPCS). Les coordonnées du centre du secteur sont encodées dans notre enregistrement du Code Postal International (IPCS).

L'ordre dans lequel les bits sont écrits est très important, car, arrondie et présentée selon notre convention, la coordonnée ayant la plus grande valeur numérique est 360.99975, 180.99975. En la convertissant en système binaire, on obtient le nombre :

$$10110100010110100111110011111110011111112 = 1\,549\,224\,509\,055_{10}$$

Le nombre obtenu est inférieur au nombre maximal du code en base 34, soit $ZZZZZZZZ_{34} = 1\,785\,793\,904\,895_{10}$. Changer l'ordre des bits pourrait faire dépasser cette valeur maximale, ce qui rendrait impossible la création de codes postaux pour tous les secteurs.

Pour retrouver les coordonnées du centre du secteur (Code Postal International), nous effectuons l'opération dans l'ordre inverse.

Exemples :

Nous construisons le Code Postal International (IPCS) pour les coordonnées -1.347120, 53.983489 comme suit :

Nous arrondissons, convertissons les coordonnées et obtenons 359.34700, 53.98350

Nous convertissons chaque élément des coordonnées — c'est-à-dire les nombres 359, 347, 00, 53, 983, 50 — en binaire comme décrit ci-dessus, et obtenons : 101100111_2 , 0101011011_2 , 00_2 , 01101010_2 , 1111010111_2 , 10_2

Nous disposons les nombres dans le bon ordre : 101100111_2 , 0011010_{21} , 0101011011_2 , 1111010111_2 , 00_2 , 10_2

Nous formons le nombre $10110011100110101010101101111110101110010_2$

Nous convertissons le nombre binaire en base 34 et obtenons $VCPM6TKY_{34}$

Le nombre obtenu est le Code Postal International (IPCS) : VCPM 6TKY

Les coordonnées du Code Postal International 1GL8 YT7F sont calculées comme suit :

Nous convertissons le nombre en base 34 $1GL8YT7F_{34}$ en binaire et obtenons $000010010001100101100000011010011011010012$

Nous extrayons les nombres 00001001_2 , 0011001_{20} , 1100000011_2 , 0100110110_2 , 10_2 , 01_2

Nous changeons l'ordre en 00001001_2 , 1100000011_2 , 10_2 , 0011001_{20} , 0100110110_2 , 01_2

Nous convertissons les nombres binaires en décimal : 18, 771, 50, 50, 310, 25

À partir des nombres obtenus, nous obtenons les coordonnées 18.77150, 50.31025

Fonction de génération et de calcul des coordonnées à partir du Code Postal International

Ci-dessous figurent deux fonctions écrites en C#. La première construit le Code Postal International, la seconde calcule les coordonnées à partir du Code Postal International.

```
public string GeneratePostcode(double Lon, double Lat)
{
    if ((Lon > 180) || (Lon < -180) || (Lat > 90) || (Lat < -90)) return "";
    if (Lat > 89.99975) Lat = 89.99975;
    if (Lat < -89.99975) Lat = -89.99975;

    string[] Coder = { "0", "1", "2", "3", "4", "5", "6", "7", "8", "9", "A", "B", "C", "D", "E", "F", "G", "H", "J",
        "K", "L", "M", "N", "P", "Q", "R", "S", "T", "U", "V", "W", "X", "Y", "Z" };

    if (Lat < 0)
    {
        Lat = ((long)Lat + 180) + Math.Abs(Lat - (long)Lat);
    }
    if (Lon < 0)
    {
        Lon = ((long)Lon + 360) + Math.Abs(Lon - (long)Lon);
    }

    string PartPostcodeLat = Lat.ToString("000.000000");
    string PartPostcodeLon = Lon.ToString("000.000000");

    try
    {
        string[] StrsLat = PartPostcodeLat.Split('.');
        string[] StrsLon = PartPostcodeLon.Split('.');

        ulong OptionsLat = 5;
        long Precision = Convert.ToInt32(StrsLat[1].Substring(3));
        if (Precision < 125)
        {
            OptionsLat = 0;
        }
        else if (Precision < 375)
        {
            OptionsLat = 1;
        }
        else if (Precision < 625)
        {
            OptionsLat = 2;
        }
        else if (Precision < 875)
        {
            OptionsLat = 3;
        }

        ulong TempHiLat = Convert.ToUInt64(StrsLat[0]);
        TempHiLat = TempHiLat << 24;
        ulong TempLoLat = 0;
        if (StrsLat.Length > 1)
        {
            TempLoLat = Convert.ToUInt64(StrsLat[1].Substring(0, 3));
            if (OptionsLat == 5)
            {
                TempLoLat++;
            }
            else
            {
                {

```

```

        TempHiLat = TempHiLat | OptionsLat;
    }

    TempLoLat = TempLoLat << 4;
}

ulong OptionsLon = 5;
Precision = Convert.ToInt32(StrsLon[1].Substring(3));
if (Precision < 125)
{
    OptionsLon = 0;
}
else if (Precision < 375)
{
    OptionsLon = 1;
}
else if (Precision < 625)
{
    OptionsLon = 2;
}
else if (Precision < 875)
{
    OptionsLon = 3;
}

ulong TempHiLon = Convert.ToUInt64(StrsLon[0]);
TempHiLon = TempHiLon << 32;
ulong TempLoLo = 0;
if (StrsLon.Length > 1)
{
    TempLoLo = Convert.ToUInt64(StrsLon[1].Substring(0, 3));
    if (OptionsLon == 5)
    {
        TempLoLo++;
    }
    else
    {
        TempHiLon = TempHiLon | (OptionsLon * 4);
    }
}

TempLoLo = TempLoLo << 14;
}

ulong UlongResHi = TempHiLat | TempHiLon | TempLoLat | TempLoLo;

ulong Pos = 0;
string Res = "";
for (int i = 7; i >= 0; i--)
{
    Pos = UlongResHi / (ulong)Math.Pow(34, i);
    UlongResHi -= (Pos * (ulong)Math.Pow(34, i));
    Res += Coder[Pos];
}

return Res;
}
catch
{
    return "";
}
}

public bool ReversePostcode(string Postcode, out double Lat, out double Lon)
{
    if (Postcode.Length == 8)
    {

```

```

Postcode = Postcode.ToUpper();
string Coder = "0123456789ABCDEFGHIJKLMNOPQRSTUVWXYZ";
ulong Hi = 0;

for (int i = 0; i < 8; i++)
{
    Hi += (ulong)Coder.IndexOf(Postcode[i]) * (ulong)Math.Pow(34, (7 - i));
}

ulong OptionLat = (Hi & 3);
double HalfLat = 0;
switch (OptionLat)
{
    case 1:
        HalfLat = 0.00025;
        break;
    case 2:
        HalfLat = 0.0005;
        break;
    case 3:
        HalfLat = 0.00075;
        break;
}

ulong OptionLon = (Hi & 12) >> 2;
double HalfLon = 0;
switch (OptionLon)
{
    case 1:
        HalfLon = 0.00025;
        break;
    case 2:
        HalfLon = 0.0005;
        break;
    case 3:
        HalfLon = 0.00075;
        break;
}

long LoLat = ((long)Hi >> 4) & 1023;
long HiLat = ((long)Hi >> 24) & 255;
if (HiLat > 180)
{
    Lat = -180;
    Lon = -180;
    return false;
}
if (HiLat > 90)
{
    HiLat = HiLat - 180;
    Lat = HiLat - ((LoLat / 1000.0) + HalfLat);
}
else
{
    Lat = HiLat + ((LoLat / 1000.0) + HalfLat);
}

long LoLon = ((long)Hi >> 14) & 1023;
long HiLon = ((long)Hi >> 32);
if (HiLon > 360)
{
    Lat = -180;
    Lon = -180;
    return false;
}
if (HiLon > 180)

```

```
{
    HiLon = HiLon - 360;
    Lon = HiLon - ((LoLon / 1000.0) + HalfLon);
}
else
{
    Lon = HiLon + ((LoLon / 1000.0) + HalfLon);
}

return true;
}
else
{
    Lat = 0;
    Lon = 0;
    return false;
}
}
```